
CM50270: Deep Reinforcement Learning for Walker2D

Jake Hulme joh46@bath.ac.uk	Umran Hussain auh26@bath.ac.uk	Carolina Masmela Correa cmc228@bath.ac.uk
Yousuf Mohammed yam38@bath.ac.uk	Calum Ogilvy cjo33@bath.ac.uk	James Sue-Ling jhmsl20@bath.ac.uk

This research made use of Hex, the GPU Cloud in the
Department of Computer Science at the University of Bath

1 Problem Definition

1.1 Walker2d

We apply deep reinforcement learning (DRL) methods to the Walker2d environment (Brockman et al., 2016), part of the MuJoCo suite (Todorov, Erez and Tassa, 2012a) within OpenAI Gym. Walker2d is a benchmark task simulating a bipedal robot learning to walk through torques applied to six joints connecting seven body parts. Powered by MuJoCo's physics engine, it poses a challenging continuous control problem requiring coordinated motor behaviour. The transition dynamics of this environment are fully deterministic. It has a relatively high-dimensional state and action spaces, offering a balance between complexity and feasibility, making it well-suited for evaluating advanced algorithms.

1.2 Observation Space

The observation space is a 17-dimensional continuous vector: **qpos** - 8 elements representing the positions of the robot's body parts; **qvel** - 9 elements representing the velocities of the individual body parts.

The observation space is represented by Box(-Inf, Inf, (17,), float64) where Box indicates a continuous observation space, -Inf and Inf are the lower and upper bounds for each element in observation, (17,) represents the 1D vector containing values for positions and velocities, and float64 is the 64-bit floating point data type representation for each observation.

1.3 Action Space

The action space is represented by the continuous action space Box(-1, 1, (6,), float32). Each action dimension must be between -1 and 1 which are normalized torque values applied to the robot's joints. The action vector, (6,) is a 6-dimensional vector with one value representing the torque applied to each of the 6 joints. The 6 torques control the left and right hip, knee and ankle joint. The torques then determine how the robot moves its limbs to walk, balance, or fall.

1.4 Rewards

The total reward is

$$r_t = r_{\text{healthy}} + r_{\text{forward}} - r_{\text{control}} \quad (1)$$

where

- the healthy reward, r_{healthy} , is a fixed reward of +1 which the Walker2d receives every time step that it is alive.
- the forward reward, r_{forward} , is a reward for moving forward i.e., in the positive x direction. By default, the forward reward is calculated by $\frac{dx}{dt}$.

- the control cost, r_{control} , is a negative reward which penalizes the Walker2d for taking too large actions. By default, this is calculated by $0.001 * \sum_{i=1}^n a_i^2$ where a_i is the torque applied at joint i and $n = 6$ is the number of controllable joints.

1.5 Episode End

An episode terminates in non-finite state values, or if the torso’s height or pitch exceeds default bounds (`healthy_z_range`, `healthy_angle_range`).

1.6 Problem Difficulty

Walker2d is a high-dimensional, continuous state-action space problem. Tabular reinforcement learning methods are inapplicable due to their inability to generalise across continuous inputs and the infeasibility of maintaining explicit Q-values for every state-action pair. Instead, function approximation is essential for learning policies in such settings. The complexity and coordination required in this task make it an ideal benchmark for evaluating advanced DRL methods, such as actor-critic algorithms.

2 Background

2.1 Stochastic vs Deterministic Policies

In reinforcement learning, a deterministic policy maps each state s to a specific action $\pi(s) = a$, with no randomness involved. In contrast, a stochastic policy defines a distribution $\pi(a|s)$, sampling actions with some probability, introducing randomness even in identical states (Sutton and Barto, 2018).

While both policy types are compatible with policy gradient methods, deterministic policies often perform better in high-dimensional continuous control tasks due to reduced variance and greater sample efficiency (Silver et al., 2014). This is especially relevant in environments like Walker2d, where continuous, fine-grained control is required. Thus, we restrict our focus mainly to deterministic policies in this report, but explore SAC (Haarnoja et al., 2018), a stochastic policy, for comparison.

2.2 On-Policy vs Off-Policy Methods

On-Policy algorithms, such as REINFORCE (Williams, 1992) and PPO (Schulman et al., 2017) learn from data collected using the current policy, while off-policy algorithms, such as DDPG (Lillicrap et al., 2019), TD3 (Fujimoto, van Hoof and Meger, 2018) and SAC (Haarnoja et al., 2018) learn from data collected using other policies, enabling experience replay and better sampling efficiency (Sutton and Barto, 2018).

Empirical studies have shown that on-policy algorithms like PPO perform suboptimally on Walker2d, showing slow reward improvement even after prolonged training (Zhang, 2025). In contrast, off-policy methods demonstrate better stability and scalability in continuous control settings (Lillicrap et al., 2019; Hausknecht and Stone, 2024). Across multiple benchmarks, TD3 consistently achieves superior performance in MuJoCo tasks (Lazaridis, Fachantidis and Vlahavas, 2020).

We speculate that on-policy methods fail in this environment since exploratory actions can upset the balance of the walker, which is delicate with respect to its joints, and are thus heavily penalised. Therefore, on-policy methods might result in a more conservative walker that is rewarded much less in comparison to an off-policy trained counterpart, which ignores the exploratory risk and thus converges to a more optimal policy.

2.3 From DDPG to TD3: Why TD3 is Better

DDPG introduced a deterministic actor-critic architecture with target networks and experience replay (Lillicrap et al., 2019), but suffers from Q-value overestimation and instability, due to bootstrapping the actor’s choice of optimal action. To address this, TD3 (Fujimoto, van Hoof and Meger, 2018) adds three core improvements: Clipped Double Q-Learning introduces 2 critics to reduce overestimation; Target policy smoothing adds noise to target actions for better generalisation; Delayed policy updates stabilise learning by updating the actor less frequently. Ablation studies by Fujimoto, van Hoof and Meger (2018) show that the combination of these techniques leads to significant performance gains in Walker2d over DDPG.

3 Method

3.1 DDPG

Naive value-based methods (e.g. DQN) struggle in continuous action space because finding $\arg \max_a Q(s, a)$ is inherently difficult and unstable when the action space is infinite (Lillicrap et al., 2019). Addressing this issue, Lillicrap et al. (2019) proposed DDPG (Deep Deterministic Policy Gradient) which is an off-policy, actor-critic algorithm designed for environments with continuous action spaces. It consists of four neural networks: an actor network θ^μ , which learns a deterministic policy mapping states to actions; a critic network θ^Q , which estimates the action-value function; and corresponding target networks $\theta^{\mu'}$ and $\theta^{Q'}$, which are slowly updated versions of the main networks used to stabilize training.

DDPG uses a uniform replay buffer, to store experience tuples $(s_t, a_t, r_t, s_{t+1}, d)$ from which random mini-batches are sampled to update the networks. This allows the agent to learn from a wider range of past experiences, avoiding issues arising from only learning from recent, consecutive steps. The critic network is updated using the Bellman equation, where next-state values are calculated with the target networks which helps to reduce the problem of updating towards a moving target. To ensure sufficient exploration in the continuous action space, noise is added to the action to prevent premature convergence to a suboptimal policy. DDPG’s use of off-policy learning, target networks, and experience replay makes it well-suited for high-dimensional continuous control tasks such as Walker2d.

3.2 TD3

TD3 (Twin Delayed Deep Deterministic Policy Gradient) improves upon DDPG, addressing overestimation bias and training instability. It retains the off-policy, actor-critic framework with deterministic policies, but introduces three key enhancements: Clipped Double Q-Learning, target policy smoothing and delayed policy updates.

TD3 uses six networks: an actor θ^μ ; two critics $\theta^{Q_1}, \theta^{Q_2}$; and their respective target networks $\theta^{\mu'}$, $\theta^{Q'_1}$, and $\theta^{Q'_2}$. During critic updates, it computes the target with the minimum of the two Q-values to reduce overestimation as follows:

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_i, \text{targ}}(s', a'(s'))$$

where

$$a'(s') = \text{clip}(\mu_{\theta^{\mu'}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{low}}, a_{\text{high}}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$$

This target policy smoothing (selecting $a'(s)$), prevents the policy from exploiting sharp Q-function peaks. Actor updates are delayed relative to critic updates. Fujimoto, van Hoof and Meger (2018) recommend one policy update for every two Q-function updates to promote more stable learning.

Like DDPG, TD3 uses a replay buffer and adds noise to actions to encourage exploration. Its algorithmic improvements lead to more robust performance in continuous control tasks, which motivates its inclusion in our experiments.

3.3 SAC

Soft Actor Critic (SAC) is an off-policy, actor-critic method that optimises a stochastic policy by maximizing both expected return and policy entropy, which promotes exploration thus preventing premature convergence to suboptimal policies (Haarnoja et al., 2018). The loss function is as follows:

$$L(\phi_i, \mathcal{D}) = \mathbb{E}_{(s, a, r, s', d) \sim \mathcal{D}} \left[(Q_{\phi_i}(s, a) - y(r, s', d))^2 \right],$$

where

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{j=1,2} Q_{\phi_{\text{targ}, j}}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}' | s') \right), \quad \tilde{a}' \sim \pi_{\theta}(\cdot | s').$$

SAC applies double clipping similar to TD3 but introduces entropy regularization, which can use either a fixed or automatically adjusted entropy coefficient for this case we used the fixed version.

We decided to implement SAC, even though research shows deterministic methods perform better in continuous control tasks, to act as a comparison and to validate this previous work.

3.4 Proportional-based Prioritised Experience Replay

If the goal is to globally reduce temporal difference (TD) errors to zero, then replaying experience by priority on its TD error – instead of randomly – makes intuitive sense. However, it remains critical to maintain sampling diversity and correct for bias in the distribution of updates to converge to the same optimal action-value function. These issues are corrected through stochastic prioritisation and applying importance sampling weights to the sampled transitions respectively (Schaul et al., 2015). The proportional-based variant incorporates the magnitude of the TD errors into the sampling choice, which prioritises based on how much might be learned from a given experience, as opposed to the rank-based variant which is lossy in this respect. Applied to actor-critic methods, it improves the rate at which the critic becomes a better judge of the actor’s decisions, thus allowing the actor to select more valuable actions, and has been shown to be successful by Hou et al. (2017).

3.5 Noisy Networks

Deterministic policy gradient methods like DDPG and TD3 use random noise added to actions sampled from the learned policy for exploration. This is an unprincipled way of exploration and tends to be sample inefficient as the noise added to the action is entirely decorrelated with the state. NoisyNet addresses this by adding parameterized noise to the weights and biases of the layers of a neural network, where the parameters scaling the noise are also updated via gradient descent with respect to the desired loss function (Fortunato et al., 2018). This results in a principled process that granularly and independently perturbs the parameters of a neural network by noise injection, while simultaneously learning the noise scales for each parameter (Fortunato et al., 2018). By modelling policy functions as NoisyNets, we get complex and state-dependent variation in the policy, potentially leading to more efficient exploration. We repurpose the policy network of our TD3 agent as a NoisyNet to investigate its effect on the sample efficiency of TD3.

4 Results

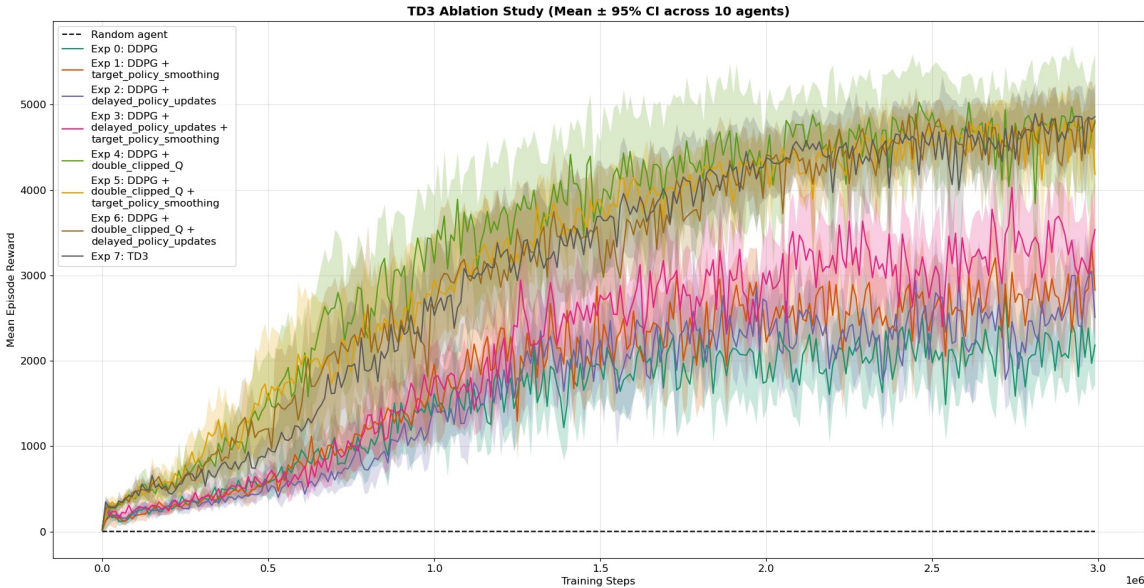


Figure 1: TD3 ablation study over 3 improvements transitioning from DDPG to TD3

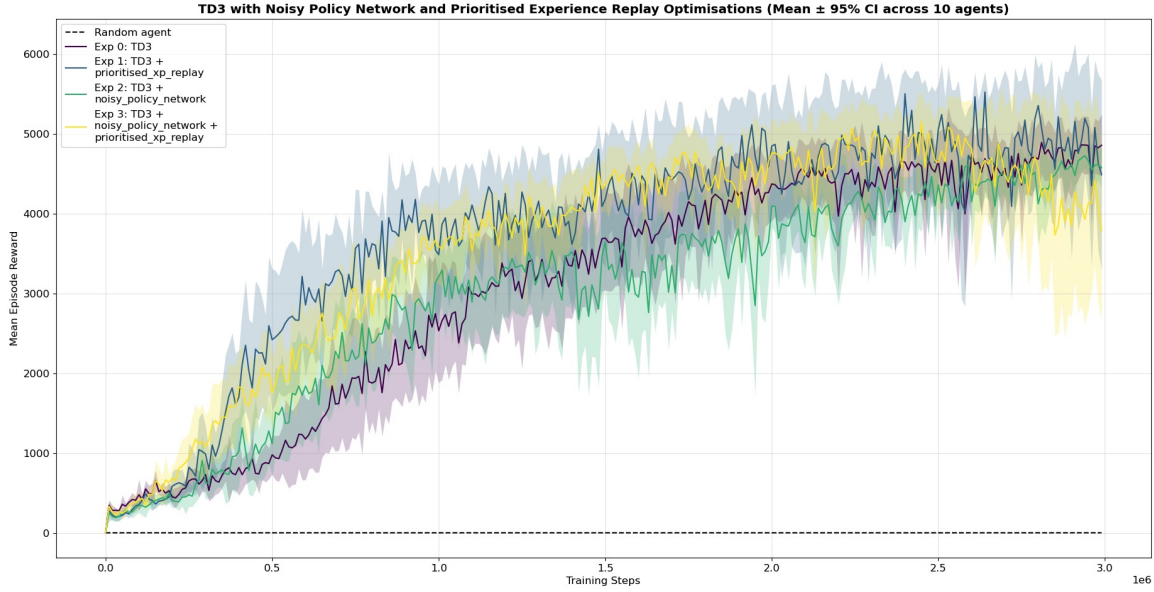


Figure 2: *TD3 with Noisy Policy Network and Prioritised Experience Replay optimisations*

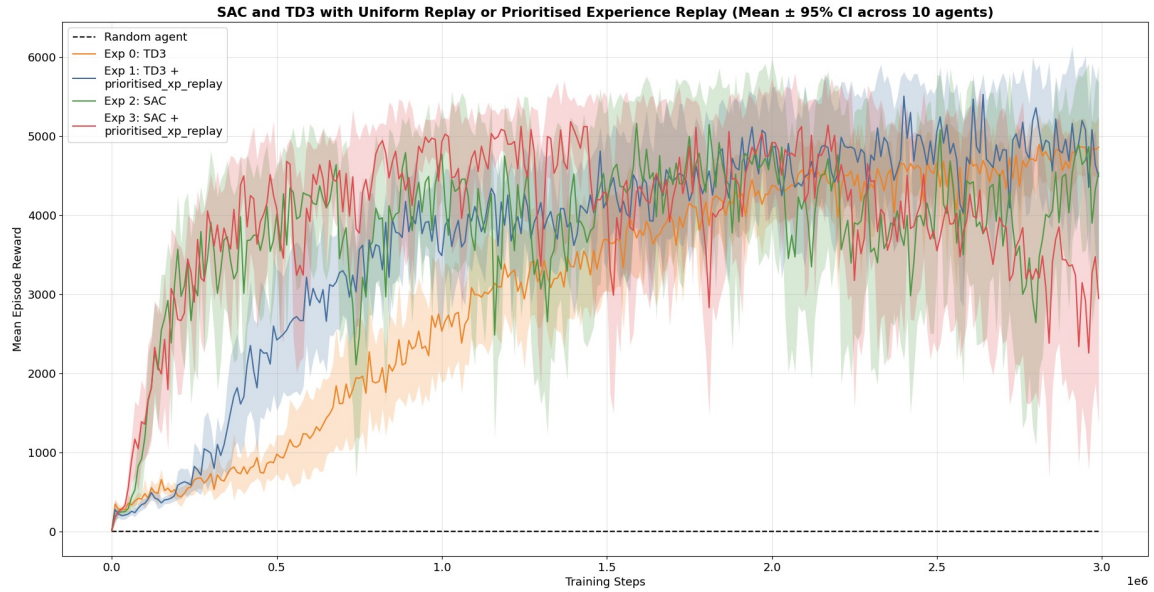


Figure 3: *SAC and TD3 compared against one another with either a uniform replay buffer or a prioritised experience replay buffer*

We performed a TD3 ablation study on the 3 improvements transitioning from DDPG to TD3 resulting in a 2^3 experiment design (Figure 1). We trained 10 agents for each experiment over 3 million time steps. Every 10,000 training steps we evaluated the performance of each agent by copying the policy network weights and measuring the agent’s mean episode reward over 10 episodes. The solid lines in Figure 1 show the average mean across the 10 agents at each evaluation point. The shaded regions show the 95% confidence interval of the population mean performance of agents in each experiment. An agent selecting actions randomly is included for comparison.

Next, we conducted an experiment to investigate whether targeted optimisations to TD3 might improve our agent’s performance (Figure 2). The optimisations were: replacing Gaussian

exploration noise in action selection with a noisy policy network; and a prioritised experience replay buffer replacing the uniform replay buffer. Finally, we compared SAC and TD3 with and without prioritised experience replay optimisation only (Figure 3). The Shapley values were then calculated for our final agent (TD3) and displayed in a ‘violin-shaped’ plot, summarising feature importance with feature effects (Figure 4).

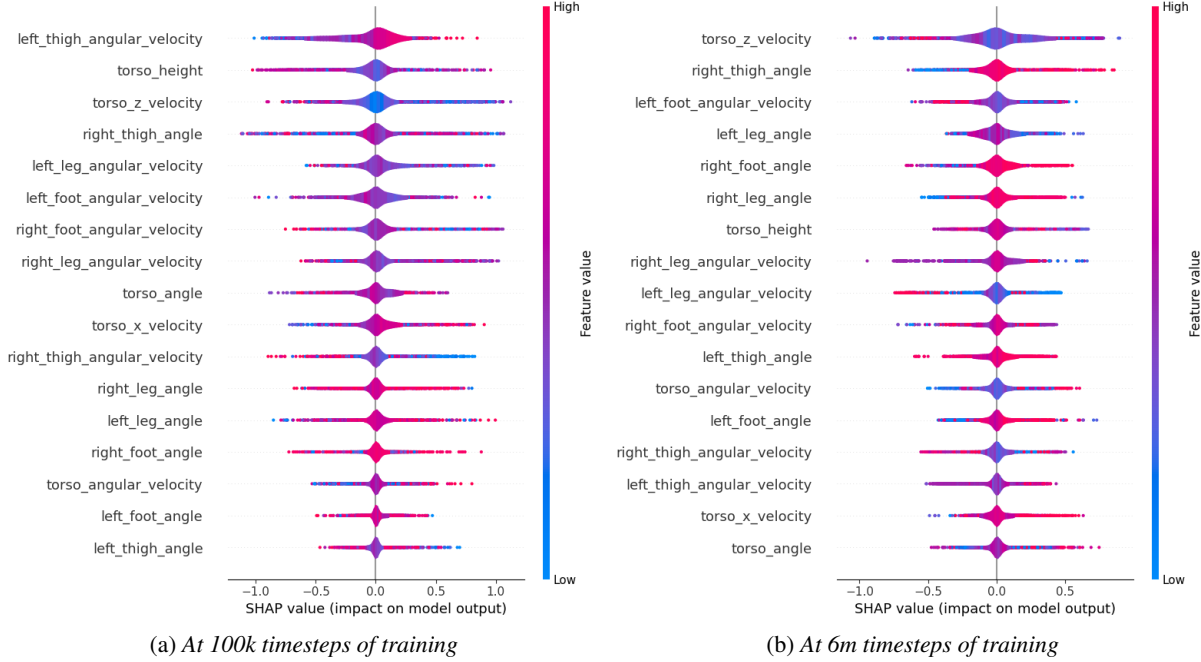


Figure 4: Feature Importance with Feature Effects

5 Discussion

Double Clipped Q-networks (DCQ) seems to be the dominant improvement (Experiments 4-7) among the 3 improvements shown in Figure 1: DCQ, Target Policy Smoothing (TPS) and Delayed Policy Updates (DPU). Additionally, these results might suggest that the combination of DCQ, with TPS or DPU, might be suboptimal. It could be that introducing an additional Q-network sufficiently corrects for variance in Q-value estimation, necessary for learning stability. Therefore, perhaps TPS simply introduces imprecision in the bootstrapped target action and DPU reduces the actor’s rate of learning here. The Walker2d environment having deterministic transition dynamics (Todorov, Erez and Tassa, 2012b) could also be a contributing factor in reducing variance in function approximation which TPS and DPU were initially designed to counteract (Fujimoto, van Hoof and Meger, 2018).

Figure 2 indicates that the Prioritised Experience Replay (PER) optimisation had a significant effect on the learning rate of the TD3 agents, with more than double the average mean episode reward over the baseline TD3 at 500,000 training steps. By ordering the replayed transitions such that the critic had more to learn from each update, this led to a more accurate critic in fewer updates which then resulted in a more informed actor. TD3 with the noisy policy network initially appears to outperform TD3 for the first 1 millions steps, but eventually gets dominated by TD3, ultimately settling on a slightly lower mean episode reward. The result from the combination of PER and noisy policy network (NPN) is surprising for us (Update: see Appendix B). We hypothesised that the combination would be purely additive in increasing performance since the PER optimisation is concerned with efficiency in experience selection whereas NPN is concerned with efficiency in experience generation, exploring the action space in an ordered manner.

The performance of SAC (Figure 3) was quite unexpected as prior literature informed us that it would not perform well in this domain. The SAC agents have an incredible learning performance over the first 500,000 training steps but convergence stability drastically deteriorates after 2 million

steps. PER appears to have a stabilising effect on SAC’s performance over the first 1.5 million training steps. In general, the SAC agents show far greater variation in their performance which supports the reasoning found from the literature.

At 100k time-steps, Figure 4a shows the ‘left_thigh_angular_velocity’ is the dominant driver, with a very wide SHAP distribution ($\approx \pm 1$). This implies that the agent watches the angular speed of the left thigh and kicks back when that velocity threatens to tip the agent, creating counter-torque in order to balance. All other features have relatively centrally located, narrow violins, suggesting they have little impact on the agent’s decisions.

By 6m time-steps the ‘torso_z_velocity’ now has the biggest mean SHAP value with a violin centred around +0.4/-0.4 (Figure 4b). This indicates that it has learnt a lower-variance strategy that uses the body’s vertical momentum as the primary trigger to determine when to ‘step’. Generally the agent then focuses on the angle of its limbs, shaping its stride length and posture before dealing with the angular velocity features to act as quick reflexes, dampening the impacts of miscalculations in the angle and fine-tuning balance. This general hierarchy of torso-vertical momentum, limb pose, joint-velocity damping follows what is used in model-based humanoid robotics (Luo et al., 2019), confirming that our agent has converged on a recognised, physics-grounded strategy for walking.

6 Future Work

6.1 Exploration

We initially tried Ornstein–Uhlenbeck noise for sufficient exploration in the policy’s action selection however it did not yield results over Gaussian noise. Our motivation for implementing noisy networks was to replace the Gaussian exploratory noise which aimed to solve the challenge of exploration for DDPG/TD3. We thought this was a weakness that might be addressed through a noisy policy (actor) network that would adaptively control noise added to weights and biases. While this yielded results over the TD3 baseline initially, it ultimately appears to converge to a lower mean episode reward. For future work, we suggest that a noisy critic network in combination with a noisy actor network might perform better.

6.2 Hyperparameter optimisation

Most of our hyperparameters were taken from OpenAI’s SpinningUp website (Achiam, 2018). Therefore, they might not necessarily be optimised for solving the Walker2d environment. Especially with the addition of the prioritised experience replay buffer, there are many tunable hyperparameters and interactions between them that might lead to greater performance in this domain. For example, it is possible to change: the linear β -schedule, which changes the rate at which the bias is corrected for in the distribution of updates; or α which affects how much to prioritise experience based on its TD error, with 0 being the case of uniform sampling and 1 being fully greedy prioritisation. We also partially experimented with clipping the priority values on entry to the buffer, which would then prioritise all transitions equally above a certain threshold, thus aiming to improve sampling diversity (see Appendix A). Initial hyperparameter searches were not promising but warrant further investigation for completeness.

7 Personal Experience

For those that were new to using PyTorch, they found it quite unforgiving in the way that broadcasting can lead to silent errors which are very difficult to track down and can cascade to affecting the entire experiment. Overall, we collectively agreed that if we were to develop code using PyTorch again, or any other numerical programming libraries that use broadcasting, it would be wise to make assertions on the shape of tensor that a function is intended to accept or output to avoid this issue e.g. `assert a.shape == (128, 1), f"Unexpected shape: {a.shape}"`. Additionally, there were some vectorised operations that some team members found quite unintuitive, also leading to bugs, such as `values[indices] += diffs`, which overwrites the values if there are duplicate indices instead of summing, as one might reasonably expect. Although tedious, in future we think it would be better to test after smaller increments, and especially when using unfamiliar libraries.

References

- Achiam, J., 2018. Spinning Up in Deep Reinforcement Learning [Online]. Available from: <https://spinningup.openai.com/en/latest/index.html#>.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J. and Zaremba, W., 2016. Openai gym. <https://gym.openai.com/>. Walker2D environment.
- Fortunato, M., Azar, M.G., Piot, B., Menick, J., Hessel, M., Osband, I., Graves, A., Mnih, V., Munos, R., Hassabis, D., Pietquin, O., Blundell, C. and Legg, S., 2018. Noisy networks for exploration. *International conference on learning representations* [Online]. Available from: <https://openreview.net/forum?id=rywHCPkAW>.
- Fujimoto, S., Hoof, H. van and Meger, D., 2018. Addressing Function Approximation Error in Actor-Critic Methods [Online]. 1802.09477, Available from: <https://doi.org/10.48550/arXiv.1802.09477> [Accessed 2025-03-29].
- Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S., 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor [Online]. 1801.01290, Available from: <https://doi.org/10.48550/arXiv.1801.01290> [Accessed 2025-03-29].
- Hausknecht, M. and Stone, P., 2024. Deep reinforcement learning in parameterized action space [Online]. 1511.04143, Available from: <https://arxiv.org/abs/1511.04143>.
- Hou, Y., Liu, L., Wei, Q., Xu, X. and Chen, C., 2017. A novel ddpq method with prioritized experience replay. *2017 IEEE international conference on systems, man, and cybernetics (smc)*. IEEE, pp.316–321.
- Lazaridis, A., Fachantidis, A. and Vlahavas, I., 2020. Deep Reinforcement Learning: A State-of-the-Art Walkthrough. *Journal of artificial intelligence research* [Online], 69, December, pp.1421–1471. Available from: <https://doi.org/10.1613/jair.1.12412> [Accessed 2025-03-29].
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. and Wierstra, D., 2019. Continuous control with deep reinforcement learning [Online]. 1509.02971, Available from: <https://doi.org/10.48550/arXiv.1509.02971> [Accessed 2025-03-29].
- Luo, J., Su, Y., Ruan, L., Zhao, Y., Kim, D., Sentis, L. and Fu, C., 2019. Robust bipedal locomotion based on a hierarchical control structure. *Robotica* [Online], 37(10), p.1750–1767. Available from: <https://doi.org/10.1017/S0263574719000237>.
- Schaul, T., Quan, J., Antonoglou, I. and Silver, D., 2015. Prioritized experience replay. *arxiv preprint arxiv:1511.05952*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal Policy Optimization Algorithms [Online]. 1707.06347, Available from: <https://doi.org/10.48550/arXiv.1707.06347> [Accessed 2025-03-29].
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D. and Riedmiller, M., 2014. Deterministic policy gradient algorithms. In: E.P. Xing and T. Jebara, eds. *Proceedings of the 31st international conference on machine learning* [Online]. Beijing, China: PMLR, *Proceedings of machine learning research*, vol. 32, pp.387–395. Available from: <https://proceedings.mlr.press/v32/silver14.html>.
- Sutton, R.S. and Barto, A.G., 2018. *Reinforcement learning: An introduction*. Cambridge, MA, USA: A Bradford Book.
- Todorov, E., Erez, T. and Tassa, Y., 2012a. Mujoco: A physics engine for model-based control. *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, pp.5026–5033.
- Todorov, E., Erez, T. and Tassa, Y., 2012b. *Mujoco: Simulation* [Online]. Available from: <https://mujoco.readthedocs.io/en/stable/programming/simulation.html> [Accessed 2025-04-30].

- Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* [Online], 8(3), May, pp.229–256. Available from: <https://doi.org/10.1007/BF00992696> [Accessed 2025-03-29].
- Zhang, J., 2025. A survey of reinforcement learning techniques for 2d and 3d bipedal locomotion. *Medium* [Online]. Available from: https://medium.com/@jer_zhang/a-survey-of-reinforcement-learning-techniques-for-2d-and-3d-bipedal-locomotion-2f82f455396a.

A Prioritised Replay Buffer Analysis

Prioritised Experience Replay Buffer Diagnostics

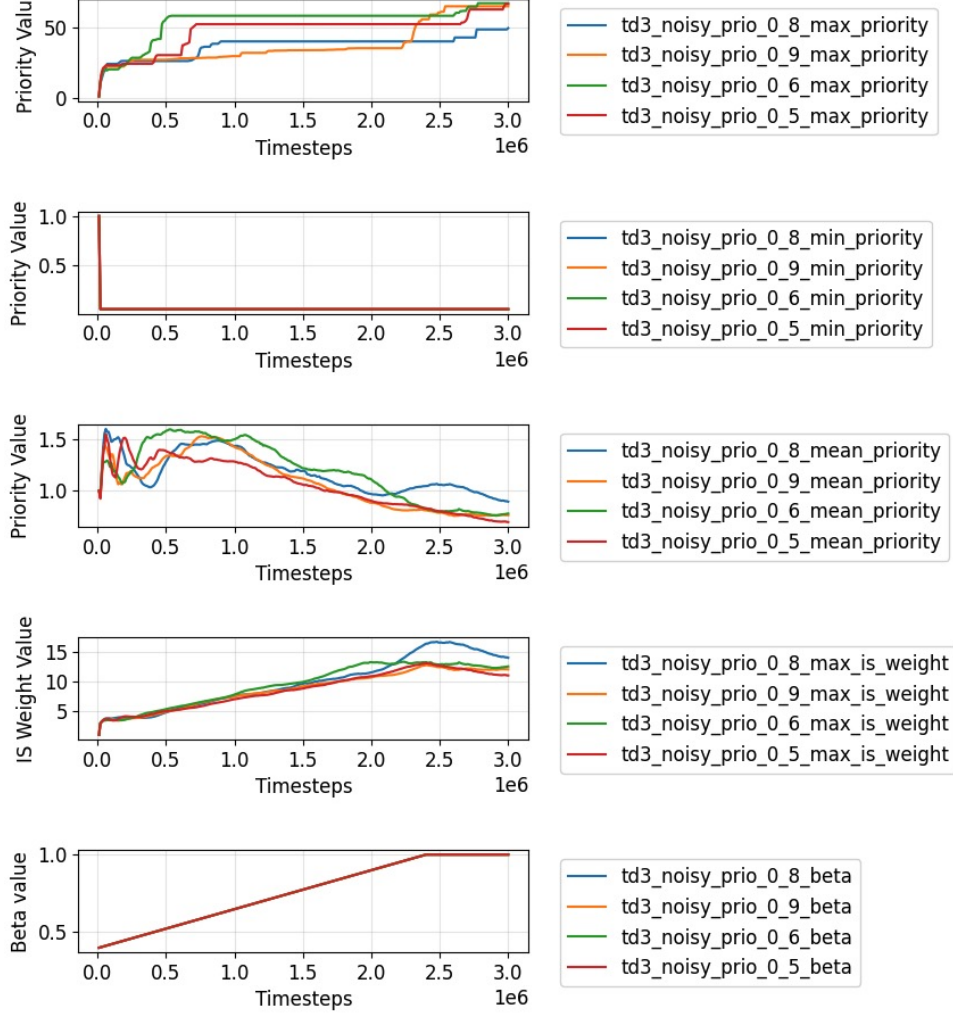


Figure 5: Metrics used for debugging the Prioritised Replay Buffer from a sample of experiments.

In Figure 5 we can see that the initial priorities of 1.0 given to the first 10,000 transitions is correctly updated downwards once they are sampled, which leads to a priority recalculation. Additionally, the linear β -schedule can be seen to be working as intended in all experiments. Over the entire duration, the buffer holds transitions with priority values of approximate mean 1 and approximate maximum 50. This shows that the priority values appear to not be excessively diverging but might be an indication of reduced sampling diversity. It could be that some of the high priority value transitions are unlearnable in the sense that our critic’s value estimation is always way off target. For example, these experiences could be pivotal or complex balancing actions performed by the walker. This is our reasoning for suggesting clipping priorities as further work.

B Hidden interaction between Noisy policy network and Prioritised Experience Replay

We did not realise until very late that there was a hidden interaction between our implementations of prioritised experience replay and the noisy policy network. This meant that it was too late to run another full experiment. In the calculation of TD errors required for the priority recalculation of the sampled transitions, a noisy policy action was used to estimate the optimal action in the next state to bootstrap off of instead of a policy action without noise. The policy action was therefore:

$$a'(s') = \text{clip}(\mu_{\theta\mu'_{noisy}}(s') + \text{clip}(\epsilon, -c, c), a_{low}, a_{high}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$$

instead of:

$$a'(s') = \text{clip}(\mu_{\theta\mu'_{actual}}(s') + \text{clip}(\epsilon, -c, c), a_{low}, a_{high}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$$

This meant that the priorities also had parametric noise injected into them (alongside smoothing noise), since they are calculated as $|\delta| + \epsilon$, where ϵ is a small positive constant, which would disrupt the prioritisation process. Seeing that the performance of the combined optimisations with TD3 is already strong (Figure 2), correcting this might have yielded even stronger results. In Figure 6 are the results from 7 out of 10 agents that we were able to train given the time.

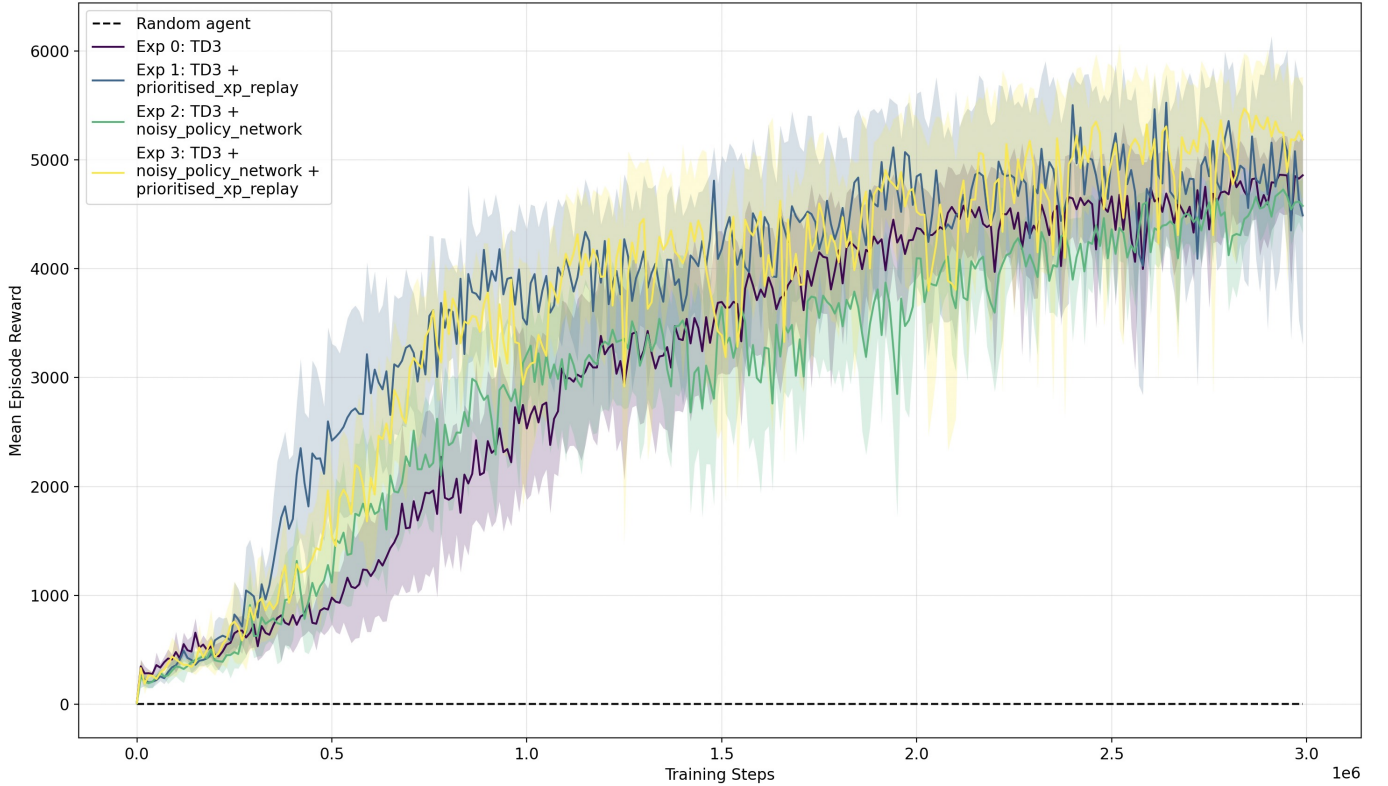


Figure 6: *TD3 with Noisy Policy Network and Prioritised Experience Replay optimisations. Please note that Experiment 3 has only 7 out of 10 agents*